



Case Study: Frontier Lab Intelligence

AI Engineer (m/f/d) · BIT Capital · take-home case study

The context

You're joining the AI Engineering team at BIT Capital, a tech-focused investment fund. We invest in technology companies, including those building frontier AI. Our edge depends on knowing what the labs are actually doing, often before it shows up in a product or a stock price.

The labs that build frontier AI (OpenAI, Anthropic, DeepMind, and the stealth startups their people leave to found) broadcast a lot of signals in public. The labs publish research blogs, model and system cards, release notes, and posts from their org accounts on X. Their researchers and engineers publish papers, technical blogs, GitHub repos, conference talks, and posts of their own. Buried in that firehose are genuinely valuable early signals: a capability breakthrough months before it's productized, a key researcher quietly moving to a competitor, a shift in a lab's strategic direction, an architectural innovation with ramifications across the semiconductor value chain.

We want you to build the system that tracks this.

Before you write any code, research BIT Capital using whatever is public: our website, fund documents, press, LinkedIn. Part of the task is figuring out why this intelligence matters to a fund like us, and who would actually use it.

Your task

Build a working system that:

- Tracks a register of who and what to follow: the frontier labs themselves, via their official channels, and the key individuals inside them.
- Ingests what they publish: lab blogs and accounts, papers, posts, repos, talks.
- Extracts, scores, and ranks structured insights from that stream, and separates signal from noise.
- Delivers reports and alerts, tailored to two internal audiences.

Use any AI coding tools you want. We expect you to, and we'll ask how you worked. As you build, keep track of where you bet on an agent or an LLM versus deterministic code, and why. That reasoning belongs in your design document.

The two audiences

The same intelligence serves two internal consumers. Build one shared core (ingestion → register → extraction → scoring) and two tailored last-mile outputs.

- Investment team (PMs and analysts): what does this mean for our positions? A star researcher leaving to found a startup; a capability jump that threatens a holding's moat or validates a thesis; shifts in the competitive map of the public companies exposed to AI, including the semiconductor and energy supply chains. Frame it in implications, tickers, and theses. Most labs are private, so part of the judgment is connecting lab developments to where they land for a public-equity investor.
- AI team (the people you'd be joining): what should we adopt or investigate? A new orchestration or evaluation technique for our own platform; a cheaper or better open model; a paper that changes how we'd build a pipeline. Frame it technically.

We want to see that you understand the difference and tailor the output, not that you build two separate systems.

The two deliverables

1. The app/system. A working system we can actually use. Best is a live deployment we can open from a link. Otherwise, a Git repo (a repo is a must, not a zip) whose README takes us from clone to running app, with all relevant data included, in a few commands. Include the database schema and real data.

2. The system design document. The document a senior engineer would write for this system, in maximum 10 pages:

- The architecture and how data flows through it.
- The key decisions, with the trade-offs and the alternatives you considered and rejected.
- Model choices: which model for which task and why, fallbacks, and where you chose an agent over deterministic code.
- Evaluation: how you measured extraction quality, controlled hallucination, and validated the scoring, including your ground-truth approach.

- Cost: approximate token usage and dollars per workflow, and how cost shaped your choices.
- How you worked: your own agentic setup, what your agents verified themselves, where the loop broke down.
- What works, what you'd do next, and the 3–5 most interesting real insights the system surfaced. This is your proof that it works.

We read this document as carefully as your code. Make every section carry a decision, its rationale, and its consequence. Clear and specific beats long and complete. Generated filler is a red flag: if a paragraph could have been written without building the system, cut it. Keep full prompts in the repo; the document explains the rationale behind how they were written.

One warning: we will not read a document that is blatantly AI-written. Use AI to help you write it – we do – but you are the author, and the result has to read like it: clear, well-organized, and intelligible to a busy human.

Optional: a short video demo.

What to build

1. The register of labs and individuals. The foundation: everything downstream is only as good as who and what you track. Track the labs as first-class entities via their official channels (research blog, org X account, publications index, GitHub org, model and system cards); a lot of the highest-value signal is published by the lab, not a person. Track the key individuals: founders and research leads, but also the layer below them, where much of the real signal sits. Decide who counts as "key" and why. Do the entity resolution: an X handle, an arXiv author name, and a GitHub account may be the same person, and a personal post is a different kind of source than an official lab post.

2. Ingestion. Pull from heterogeneous sources on a schedule: lab channels, arXiv, GitHub, X, and more. The sources are messy, and some are gated or expensive. Handle source selection, rate limits, deduplication, and freshness. A thoughtfully scoped set of sources done well beats a broad set done badly; justify your coverage.

3. Extraction. For each tracked entity, extract the key contributions and developments over a recent window (~3 months is a good default; recency and trend matter). Turn raw content into structured insights, attributed to the person or the lab, with the LLM doing the heavy lifting. Every insight must cite its primary source.

4. Scoring. Score both contributors and insights. A score is only as good as its justification: what are the inputs, why those weights, how do you know it measures what you claim? Validate it against ground truth, human judgment, or a defensible proxy. An arbitrary weighted sum dressed up as a score is not useful; a simple model you can defend and have tested is exactly what we want. Keep this part lean and spend the time you save on the system itself.

5. Signal vs. noise. Most of the firehose is noise. Surface the genuinely important, novel, and actionable, and suppress the rest. This is where taste shows. The single most important question we'll ask when we open your submission: did it surface something we'd genuinely want to know, and did it keep the noise out?

6. Reports and alerts. A periodic digest (you choose the cadence) readable in the app, and alerts when something material appears. The judgment is in the alerting logic: what counts as material, and for whom. Delivery into Slack or email is optional; an alert surfaced clearly in the app is enough. Both outputs must cite sources and be tailored per audience. "Actionable" means a reader knows what it means and what to do, not just what happened.

7. Web interface. A small surface to browse the register, see the scored insights and why they were flagged, and read past reports. Configuring what's tracked doesn't need a UI; config in code or files is fine. Keep it light, and don't spend much time polishing.

How we evaluate

The case is deliberately ambitious. We set a high ceiling on purpose, because we want to see how far you can take it. Aim to complete it all; if you can't, depth beats coverage. When we open your submission, we look for three things, in order:

1. A well-engineered system. Real engineering, end to end: sound architecture, a well-designed database and schema, clean modular code, and sensible infrastructure, plus deliberate machinery around the agents: model and tool choices you can explain, and handling for the failure modes you expect.

The bar to beat: engineer it like a production system you would be on call for. Robust, reliable, monitored, alerting you when something breaks, and built on abstractions and primitives that make it easy to extend. What each of those means for this system is yours to work out; that judgment is part of what we are hiring.

2. An actually useful, usable product. Did it surface something we'd genuinely want to know, and did it keep the noise out? The core skill is putting yourself in the users' shoes: picture the PM and the AI engineer actually working with this, anticipate what they need, and build what makes them productive. Then use your own product the way they would. It's less about visual

polish and more about actual usefulness.

3. An insightful system design document. This is where we read your judgment: the decisions, the trade-offs, the alternatives you rejected, and why.

Engineering and product are where this role is won; we expect you to exceed the bar in at least one of them. Two supporting dimensions still have to clear the bar, though they are not where to spend your marginal hour: data science (honest ground truth and validation, hallucination under control; a system we can't trust fails regardless of its engineering) and domain (understand BIT Capital and connect technical signals to investment consequences; intelligence disconnected from markets is not useful to us).

We assess the submission and the on-site discussion together. Expect us to go deep on your design choices and ask where you cut corners and why.

Logistics

- Deadline: check the email for the submission deadline.
- Budget: €100 for APIs and other services, and we're interested in how you spend it. Keep your receipts and send them to us; we'll reimburse up to €100.
- After submission: an on-site case discussion with the AI team. We'll go deep on your design choices, trade-offs, and what you'd build next, so come ready to discuss and extend your work.